

Runbook Hardware — Charly v2

De Bluetooth a I2S, y de ahí al primer satélite de voz.

Escrito asumiendo cero experiencia previa en electrónica. Si algo te parece obvio, salteálo.

Dónde se ejecuta cada cosa:

Símbolo	Lugar
	Trabajo físico: cables, plaquetas, soldador
	Raspberry Pi (SSH)
	Tu PC (compilar firmware del ESP32)
	Neon — SQL Editor
	n8n — UI web

Índice

- [Reglas de seguridad](#)
- [Fase 0 — Baseline y herramientas](#)
- [Fase 1 — DAC I2S en la Pi](#)
- [Fase 2 — Matar el Bluetooth y medir](#)
- [Fase 3 — Armar el satélite](#)
- [Fase 4 — Firmware push-to-talk](#)
- [Fase 5 — STT en el VPS](#)
- [Fase 6 — TTS con Piper](#)
- [Fase 7 — Wake word](#)
- [Fase 8 — Gabinete e instalación](#)

Reglas de seguridad

Nada de esto es peligroso para vos: todo trabaja a 3,3V y 5V. El riesgo es para las plaquetas.

Las cinco reglas:

1. **Siempre desenchufá la Pi antes de tocar los pines.** No es por vos, es porque un cable que toca donde no va con la placa encendida quema el pin. Apagar por software no alcanza: `sudo shutdown -h now` y después sacá el cable de alimentación.
2. **Contá los pines dos veces antes de enchufar.** Es el error más común y el más caro.

3. **3,3V y 5V no son intercambiables.** El PCM5102A acepta 5V en `(VIN)` porque tiene regulador propio. El ESP32 y los módulos de micrófono son 3,3V. Meter 5V en un pin de 3,3V lo mata.
 4. **Tocá algo metálico conectado a tierra antes de manipular las plaquetas.** Un radiador, la carcasa de una PC enchufada. Descarga la estática.
 5. **Si algo se calienta o huele raro, desenchufá.** Ninguna de estas plaquetas debería pasar de tibia.
-

Fase 0 — Baseline y herramientas

0.1 🍓 Medir el consumo actual de CPU

Este número es el que después te dice si valió la pena. Sin él, todo lo que sigue es fe.

Poné música sonando por Bluetooth como siempre, y con eso corriendo:

```
bash

top -b -n 12 -d 5 | grep -E "mpv|pipewire|bluealsa|uvicorn" > ~/baseline_bt.txt
vcgencmd measure_temp
cat /proc/loadavg
free -h
```

Miralos:

```
bash

cat ~/baseline_bt.txt
```

Anotá en algún lado el `(%CPU)` promedio de `(pipewire)` y de `(mpv)`. Ese es tu punto de comparación.

0.2 Lista de compras

#	Componente	Cant.	~USD	Fase
1	Módulo GY-PCM5102 (DAC I2S)	1	3	1
2	Cables dupont hembra-hembra 10cm	1 set	2	1
3	Cable jack 3.5mm → 2× RCA	1	3	1
4	ESP32-S3-DevKitC-1 N8R8	1	8	3
5	Microfono INMP441	1	2	3
6	Amplificador MAX98357A	1	3	3
7	Parlante 4Ω 3W	1	3	3
8	Protoboard 400 puntos	1	3	3
9	Cables dupont macho-macho	1 set	2	3
10	Fuente USB-C 5V 2A	1	5	3

Cuidado al comprar:

- **Ítem 1:** busca el que tiene **jack 3.5mm y una hilera de 11 pines con serigrafía** (VCC, 3.3V, GND, FLT, DEMP, SCK, BCK, DIN, LCK, FMT, XMT). Hay variantes sin **SCK** en el header — esas te obligan a soldar.
- **Ítem 4: S3, no S2.** La versión **N8R8** (8MB flash, 8MB PSRAM). Las de 2MB no alcanzan para los audios pre-grabados de la Fase 6.
- **Ítems 5 y 6:** suelen venir con la tira de pines **suelta, sin soldar**. Si no soldás, busca "pre-soldered" o "soldered headers" en el título, o pedí que te los suelden. Son 5 minutos de trabajo para cualquiera con soldador.

0.3 Herramientas

Fase 1 (DAC): no necesitás nada. Cero soldadura. Solo los cables dupont.

Fase 3 (satélite): si los módulos vinieron sin pines soldados, necesitás soldador de 30W con punta fina y estaño de 0,8mm. Es soldadura básica — 11 puntos por plaqueta, sin componentes delicados.

Opcional pero útil: multímetro barato (~10 USD). Sirve para verificar que un cable hace contacto antes de encender, que es la mitad del debugging de hardware.

Fase 1 — DAC I2S en la Pi

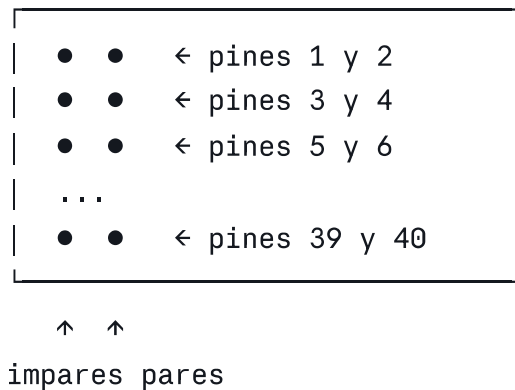
Qué vas a hacer: conectar una plaquita de 3 dólares a 5 pines de la Pi, para que el audio salga por cable en vez de por Bluetooth.

Por qué: el Bluetooth te está comiendo CPU comprimiendo audio en tiempo real, y compite con el WiFi por la misma antena. Con el DAC, la Pi manda los datos crudos por cable y un chip los convierte a analógico.

Tiempo: 30 minutos. **Soldadura:** ninguna.

1.1 🍷 Entender el header de la Pi

La Raspberry Pi tiene 40 pines en dos hileras de 20. **La numeración va en zigzag, no en línea:**



Cómo encontrar el pin 1: con la Pi apoyada y el header a la derecha, el pin 1 es el de arriba de todo en la hilera **interna** (la más cercana al borde de la placa). Muchas Pi tienen un cuadradito serigrafiado o un pad cuadrado en vez de redondo marcando el pin 1.

Truco para no equivocarte: contará los pines de a pares desde arriba. El par número N son los pines $(2N-1)$ y $(2N)$. El pin 12 está en el sexto par, hilera par.

1.2 🍷 Configurar el módulo DAC

El PCM5102A tiene un pin llamado (SCK) (system clock). Si queda al aire, el chip espera un reloj maestro que la Pi no le manda, y no sale audio. **Conectándolo a masa, el chip usa su PLL interno.**

Es el 90% de los "no me suena el PCM5102A" que vas a encontrar googleando.

Como (SCK) está en el header, se resuelve con un cable. Sin soldar.

Los otros pines de configuración ((FLT) , $(DEMP)$, (FMT) , (XMT)) vienen pre-configurados de fábrica en la mayoría de las placas. No los toques todavía.

1.3 🍷 Cablear

Apagá la Pi y desenchufala:

```
bash  
  
sudo shutdown -h now
```

Esperá que se apague el LED verde. Sacá el cable de alimentación.

Ahora conectá seis cables dupont hembra-hembra:

Módulo PCM5102A	Pin físico Pi	Qué es
VIN	2	Alimentación 5V
GND	6	Masa
SCK	9	Masa (habilita el PLL interno)
BCK	12	Bit clock
LCK	35	Left/Right clock
DIN	40	Datos de audio

Visualmente, en el header:

```
1 ● ● 2  ← VIN (rojo)  
3 ● ● 4  
5 ● ● 6  ← GND (negro)  
7 ● ● 8  
9 ● ● 10 ← SCK a masa (negro)  
11 ● ● 12 ← BCK (amarillo)  
13 ● ● 14  
...  
33 ● ● 34  
35 ● ● 36 ← LCK (verde)  
37 ● ● 38  
39 ● ● 40 ← DIN (azul)
```

Verificá dos veces antes de enchufar. Un VIN en el pin equivocado puede matar el módulo.

Notas:

- Los colores son sugerencia, no requisito. Usá rojo para 5V y negro para GND por costumbre — te va a ahorrar errores después.
- Si tu módulo dice `LRCK` en vez de `LCK`, o `BCLK` en vez de `BCK`, es lo mismo.
- Algunos módulos tienen `VCC` en vez de `VIN`. También es lo mismo.

1.4 🍷 Conectar al amplificador

El módulo tiene un jack de 3,5mm. Del jack al Fosi con el cable jack → 2× RCA:

- **Rojo** → canal derecho (R) del Fosi
- **Blanco o negro** → canal izquierdo (L)

Baja el volumen del Fosi al mínimo antes de encender. Si algo está mal configurado, el primer sonido puede ser un pop fuerte.

1.5 🍓 Configurar Raspbian

Enchufá la Pi y entrá por SSH.

```
bash
sudo nano /boot/firmware/config.txt
```

Buscá esta línea y comentala con `#`:

```
ini
#dtparam=audio=on
```

Al final del archivo agregá:

```
ini
# --- DAC I2S ---
dtoverlay=hifiberry-dac
```

`Ctrl+O`, `Enter`, `Ctrl+X` para guardar y salir.

```
bash
sudo reboot
```

El overlay se llama `hifiberry-dac` aunque tu placa no sea HiFiBerry. Es el driver genérico para DACs I2S sin control por I2C, que es exactamente lo que es el PCM5102A.

1.6 🍓 Verificar que el sistema lo ve

```
bash
aplay -l
```

Tenés que ver algo así:

```
card 0: sndrpihifiberry [snd_rpi_hifiberry_dac], device 0: HifiBerry DAC HiFi
pcm5102a-hifi-0
```

Si no aparece:

```
bash
dmesg | grep -i -E "hifiberry|i2s|pcm5102"
cat /boot/firmware/config.txt | grep -E "dtparam=audio|dtoverlay"
```

El overlay carga aunque el hardware no esté, así que si aparece en `aplay -l` solo significa que el driver está — no que el cableado esté bien. Eso lo dice el paso siguiente.

1.7 🍓 Primer sonido

Subí el volumen del Fosi a un cuarto de recorrido.

```
bash
speaker-test -D hw:0,0 -c2 -t sine -f 440 -l 1
```

Tenés que escuchar un tono continuo, primero por un parlante y después por el otro. `Ctrl+C` para cortar.

Si no se escucha nada:

Síntoma	Causa probable	Qué hacer
Silencio absoluto	<code>SCK</code> sin conectar a masa	Revisá el cable del pin 9
Silencio, <code>aplay -l</code> OK	<code>XMT</code> en masa (mute activo)	Ver 1.8
Ruido blanco o zumbido	<code>DIN</code> , <code>BCK</code> o <code>LCK</code> mal	Recontá los pines 12, 35, 40
Solo un canal	Cable RCA flojo	Revisá el jack
Sonido distorsionado	Formato mal configurado	Ver 1.8

1.8 🍷 Si hace falta ajustar jumpers

Solo si el paso anterior falló. Dá vuelta el módulo: en la parte de atrás hay cuatro pares de pads con un puente de soldadura, etiquetados `H1L`/`H2L`/`H3L`/`H4L` o `1`/`2`/`3`/`4`.

Pad	Controla	Posición correcta
H1	<code>FLT</code> — filtro digital	Normal latency
H2	<code>DEMP</code> — de-emphasis	Off
H3	<code>XSMT</code> — soft mute	Unmute (el crítico)
H4	<code>FMT</code> — formato de datos	I2S

Los módulos vienen configurados de fábrica para I2S estándar. Si tuviste que llegar hasta acá, sacale una foto a la parte de atrás y comparala con el datasheet de tu vendedor: la serigrafía varía entre fabricantes.

Alternativa sin soldar: si `XMT` está en el header, cablealo al pin `3.3V` del propio módulo con un dupont corto. Eso fuerza unmute sin tocar los pads.

1.9 🍓 Apuntar mpv al DAC

Averiguá el nombre exacto del dispositivo:

```
bash
mpv --audio-device=help | grep -i -E "hifiberry|pipewire"
```

Vas a ver algo como:

```
'pipewire/alsa_output.platform-soc_sound.stereo-fallback'
```

Editá el servicio:

```
bash
systemctl --user edit --full mpv-player
```

En la línea `ExecStart`, agregá el parámetro (usá el nombre que te devolvió el comando de arriba):

```
--audio-device=pipewire/alsa_output.platform-soc_sound.stereo-fallback
```



```
bash
```

```
systemctl --user daemon-reload  
systemctl --user restart mpv-player  
systemctl --user restart media-api
```

1.10 Probar de punta a punta

Desde Telegram, pedí una playlist. Tiene que sonar por el Fosi.

Ese es el momento en que el Bluetooth dejó de ser necesario.

Fase 2 — Matar el Bluetooth y medir

El DAC ya funciona, pero el stack de Bluetooth sigue corriendo y ocupando la antena de 2,4 GHz. Sacarlo es donde está la ganancia real.

2.1 🍓 Desactivar

```
bash
```

```
sudo systemctl disable --now bluetooth  
sudo systemctl disable --now hciuart
```

```
bash
```

```
sudo nano /boot/firmware/config.txt
```

Agregá al final:

```
ini
```

```
dtoverlay=disable-bt
```

```
bash
```

```
sudo reboot
```

Verificar que se fue:

```
bash
```

```
systemctl status bluetooth --no-pager    # inactive (dead)  
hciconfig                                # sin dispositivos
```

2.2 🍓 Medir de nuevo

Poné música y repetí exactamente lo mismo que en la Fase 0:

```
bash
```

```
top -b -n 12 -d 5 | grep -E "mpv|pipewire|uvicorn" > ~/baseline_i2s.txt  
vcgencmd measure_temp  
cat /proc/loadavg
```

```
bash
```

```
echo "=== ANTES (Bluetooth) ==="; cat ~/baseline_bt.txt  
echo "=== DESPUÉS (I2S) ==="; cat ~/baseline_i2s.txt
```

Lo que esperás: que `(pipewire)` baje de forma notable, porque ya no está comprimiendo audio a SBC en tiempo real. Y que la temperatura baje unos grados.

Ese delta es el presupuesto de CPU con el que contás para wake word, streaming de satélites y TTS.

2.3 Prueba de WiFi

Con el Bluetooth apagado, la antena de 2,4 GHz es toda para el WiFi:

```
bash
```

```
ping -c 50 1.1.1.1 | tail -3  
iwconfig wlan0 | grep -E "Signal|Bit Rate"
```

Si antes tenías cortes o latencia irregular, deberían haber mejorado.

2.4 Punto de decisión

Con el número de CPU en la mano:

- **Menos del 15% de un core:** el Pi 3B tiene margen de sobra. Olvidate del cambio de SoC por ahora, y si comercializás mirá el Pi Zero 2 W, que es más barato y alcanza.
- **Entre 15% y 40%:** margen justo. Seguí con el Pi 3B y evaluá el RK3308 cuando tengas 2 o 3 satélites funcionando.
- **Más del 40%:** algo raro pasa. Antes de comprar hardware, revisá si yt-dlp está transcodificando en vez de pasar el stream directo.

Fase 3 — Armar el satélite

Qué vas a hacer: un ESP32 con micrófono y parlante, en protoboard, que escucha cuando apretás un botón y le manda el audio a la Pi.

Poné el ESP32 a caballo del canal central. Así cada pin queda con 4 agujeros libres al costado para cablear.

3.3 🍷 Cablear

Con el ESP32 desconectado del USB.

Primero, alimentación a las tiras:

- Pin **3V3** del ESP32 → tira **+**
- Pin **GND** del ESP32 → tira **-**

Micrófono INMP441:

INMP441	ESP32-S3	Qué es
VDD	tira + (3,3V)	alimentación
GND	tira -	masa
SCK	GPIO4	bit clock
WS	GPIO5	word select
SD	GPIO6	datos hacia el ESP32
L/R	tira -	fija el canal izquierdo

Amplificador MAX98357A:

MAX98357A	ESP32-S3	Qué es
VIN	tira + (3,3V)	alimentación
GND	tira -	masa
BCLK	GPIO15	bit clock
LRC	GPIO16	word select
DIN	GPIO7	datos hacia el amplificador
GAIN	sin conectar	ganancia por defecto, 9dB
SD	sin conectar	enable, activo por pull-up interno

Parlante: los dos cables a los borneros **+** y **-** del MAX98357A. En un parlante no importa la polaridad si es uno solo — solo importa si tenés dos y querés que estén en fase.

Botón: no cablees nada. El ESP32-S3-DevKitC-1 ya trae un botón (BOOT) conectado a (GPIO0). Lo usamos como push-to-talk en la Fase 4. Una cosa menos.

3.4 🔧 Revisar antes de encender

Con el USB todavía desconectado:

1. **Ningún cable de la tira (+) toca la tira (-).** Un cortocircuito ahí puede dañar el ESP32.
2. (VDD) del INMP441 va a 3,3V, no a 5V. El módulo no tolera 5V.
3. **Ningún pin del ESP32 tiene dos cables.** Salvo las tiras de alimentación.
4. **Los módulos están bien asentados,** no torcidos ni con un pin al aire.

Si tenés multímetro, en modo continuidad: entre (+) y (-) **no** tiene que pitar. Si pita, hay un corto.

3.5 🔧 Primer encendido

Enchufá el USB-C.

- El LED del ESP32 se enciende.
- Ningún módulo se calienta.
- Nada huele a quemado.

Tocá el INMP441 y el MAX98357A con el dedo después de un minuto. Tienen que estar a temperatura ambiente. Si alguno quema, **desenchufá** y revisá que no tenga 5V donde va 3,3V.

3.6 💻 Verificar que la PC lo ve

```
bash
# Linux
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
dmesg | tail -5
```

Si no aparece, probá el otro puerto USB del ESP32 — el DevKitC-1 tiene dos: uno es USB nativo y el otro es el chip UART. Para empezar usá el que dice (UART).

Fase 4 — Firmware push-to-talk

Estrategia: botón primero, wake word después. Así validás toda la cadena (satélite → Pi → VPS → texto → comando) sin pelearte con la detección de voz.

4.1 💻 Instalar ESP-IDF

En tu PC, no en la Pi.

```
bash
```

```
sudo apt install -y git wget flex bison gperf python3 python3-venv \
    cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0

mkdir -p ~/esp && cd ~/esp
git clone -b v5.2 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf && ./install.sh esp32s3
```

Cada vez que abras una terminal nueva para trabajar con el ESP32:

```
bash

. ~/esp/esp-idf/export.sh
```

Ponelo en un alias, lo vas a escribir mucho:

```
bash

echo "alias idf='. ~/esp/esp-idf/export.sh'" >> ~/.bashrc
```

4.2 Crear el proyecto

```
bash

cd ~/esp
idf.py create-project charly-satelite
cd charly-satelite
```

Estructura mínima:

```
charly-satelite/
├─ CMakeLists.txt
├─ sdkconfig.defaults
└─ main/
    ├─ CMakeLists.txt
    ├─ main.c
    ├─ config.h
    ├─ audio.c / audio.h
    └─ net.c / net.h
```

4.3 Configuración

main/config.h:

```
c
```

```
#pragma once

// --- Red ---
#define WIFI_SSID      "tu-red"
#define WIFI_PASS      "tu-password"
#define PI_HOST        "192.168.0.XX"    // IP fija de la Pi
#define PI_PORT        8001
#define ROOM_ID        "cocina"

// --- Micrófono (I2S_NUM_0) ---
#define MIC_SCK        4
#define MIC_WS         5
#define MIC_SD         6

// --- Parlante (I2S_NUM_1) ---
#define SPK_BCLK        15
#define SPK_LRC         16
#define SPK_DIN         7

// --- Botón ---
#define BOTON_GPIO      0    // BOOT integrado en la placa

// --- Audio ---
#define SAMPLE_RATE     16000
#define CHUNK_SAMPLES   320    // 20ms
```

El ESP32-S3 tiene **dos controladores I2S independientes**. Micrófono y parlante van cada uno al suyo. Ese problema del "bloque único" que tenía la Pi acá no existe.

4.4 Flashear y probar

```
bash

idf.py set-target esp32s3
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

[Ctrl+J] para salir del monitor.

Prueba de micrófono aislada: antes de meter red, hacé que el firmware imprima el nivel RMS del audio capturado en la consola serie. Si al hablarle sube y al callarte baja, el micrófono anda. Si es siempre 0 o siempre saturado, revisá **[L/R]** a masa y el cableado de **[SD]**.

4.5 🍓 Endpoint en la Pi

app/satellite.py:

```
python
```



```

"""Recepción de audio desde satélites"""
import logging

from fastapi import APIRouter, WebSocket, WebSocketDisconnect

logger = logging.getLogger(__name__)
router = APIRouter()

@router.websocket("/satellite/{room_id}")
async def satellite_ws(ws: WebSocket, room_id: str):
    await ws.accept()
    logger.info("satélite conectado: %s", room_id)
    buffer = bytearray()

    try:
        while True:
            msg = await ws.receive()

            if "bytes" in msg:
                buffer.extend(msg["bytes"])

            elif "text" in msg:
                import json
                evt = json.loads(msg["text"])

                if evt.get("type") == "audio_start":
                    buffer.clear()
                    logger.info("[%s] captura iniciada", room_id)

                elif evt.get("type") == "audio_end":
                    dur = len(buffer) / (16000 * 2)
                    logger.info("[%s] %d bytes (%.1fs)", room_id, len(buffer),
                                await ws.send_json({"type": "ack"}))
                    # Fase 5: acá va el STT
                    buffer.clear()

    except WebSocketDisconnect:
        logger.info("satélite desconectado: %s", room_id)

```

En `main.py`:

```
python
```

```
from app.satellite import router as satellite_router
app.include_router(satellite_router)
```

Uvicorn en el puerto 8000 sirve WebSocket sin configuración extra, así que si preferís no abrir un puerto nuevo, usá `8000` en `config.h`.

4.6 Prueba de la cadena

```
bash
```

```
journalctl --user -u media-api -f | grep satélite
```

Apretá el botón `BOOT` del ESP32 y hablá. Tenés que ver los bytes llegando con una duración coherente con lo que hablaste.

Con eso validado, el resto es software.

Fase 5 — STT en el VPS

5.1 Instalar faster-whisper

```
bash
```

```
ssh usuario@vps
sudo apt install -y python3-venv ffmpeg
mkdir -p ~/stt && cd ~/stt
python3 -m venv .venv && source .venv/bin/activate
pip install faster-whisper fastapi uvicorn python-multipart
```

5.2 Servicio de transcripción

```
~/stt/server.py:
```

```
python
```

```

from fastapi import FastAPI, UploadFile
from faster_whisper import WhisperModel
import tempfile, logging

logging.basicConfig(level=logging.INFO)
app = FastAPI()

model = WhisperModel("small", device="cpu", compute_type="int8")

@app.post("/transcribe")
async def transcribe(audio: UploadFile):
    with tempfile.NamedTemporaryFile(suffix=".wav") as f:
        f.write(await audio.read())
        f.flush()
        segs, info = model.transcribe(f.name, language="es", beam_size=1)
        texto = " ".join(s.text for s in segs).strip()

    logging.info("transcripción: %r", texto)
    return {"text": texto, "language": info.language}

```

`beam_size=1` sacrifica algo de precisión por velocidad. Para comandos cortos de voz es el trade-off correcto.

Como servicio:

```

bash

sudo nano /etc/systemd/system/stt.service

```

```

ini

```

```
[Unit]
Description=STT faster-whisper
After=network.target

[Service]
User=TU_USUARIO
WorkingDirectory=/home/TU_USUARIO/stt
ExecStart=/home/TU_USUARIO/stt/.venv/bin/uvicorn server:app --host 127.0.0.1 --
Restart=always

[Install]
WantedBy=multi-user.target
```

bash

```
sudo systemctl daemon-reload
sudo systemctl enable --now stt
```

Escuchá solo en `127.0.0.1` y exponelo por el Nginx que ya tenés, con auth. Un endpoint de transcripción abierto a internet es una factura de CPU esperando a pasar.

5.3 🍓 Conectar en la Pi

En `satellite.py`, dentro del `audio_end`:

```
python

import httpx
from app.wav import pcm_to_wav # helper: agrega header WAV a PCM crudo

async def _transcribir(pcm: bytes) -> str:
    wav = pcm_to_wav(pcm, sample_rate=16000)
    async with httpx.AsyncClient(timeout=30) as c:
        r = await c.post(
            "https://stt.countercrm.com/transcribe",
            files={"audio": ("a.wav", wav, "audio/wav")},
            headers={"X-API-Key": settings.stt_key},
        )
    r.raise_for_status()
    return r.json()["text"]
```

Y el texto va al mismo router de n8n que ya recibe Telegram, con el `room_id` adjunto. El pipeline de comandos es exactamente el que ya tenés funcionando.

Fase 6 — TTS con Piper

6.1 🍓 Instalar

```
bash

cd ~ && mkdir -p piper && cd piper
wget https://github.com/rhasspy/piper/releases/latest/download/piper_linux_armv
tar -xzf piper_linux_armv7l.tar.gz
```

Descargá una voz rioplatense (`es_AR-daniela-high`) desde el repo de voces de Piper. Son dos archivos: el `.onnx` y el `.onnx.json`.

```
bash

echo "Hola, soy Charly" | ./piper --model es_AR-daniela-high.onnx --output_file
aplay /tmp/t.wav
```

Piper sintetiza más rápido que tiempo real incluso en el Pi 3B.

6.2 🍓 Endpoint /speak

```
python

@router.post("/speak")
async def speak(req: SpeakRequest):
    """Sintetiza y manda al satélite, o al sistema principal con ducking."""
    wav = await asyncio.to_thread(_piper, req.text)

    if req.room_id and req.room_id in _satelites:
        await _satelites[req.room_id].send_json({"type": "speak"})
        await _satelites[req.room_id].send_bytes(wav)
    else:
        if req.duck:
            await _duck(30)          # baja mpv al 30%
            await asyncio.to_thread(_reproducir_wav, wav)
        if req.duck:
            await _unduck()
    return {"ok": True}
```

6.3 🖥️ Audios pre-grabados en el satélite

Grabá con Piper y metelos en el flash del ESP32:

Archivo	Cuándo suena
<code>beep.wav</code>	wake word detectado
<code>dale.wav</code>	comando recibido
<code>listo.wav</code>	comando ejecutado
<code>no_entendi.wav</code>	STT vacío o sin match
<code>sin_conexion.wav</code>	no llega a la Pi

Con estos en local, la respuesta se siente instantánea aunque el backend tarde 1,5 segundos. Es la diferencia entre "responde rápido" y "responde bien".

Fase 7 — Wake word

Recién acá se reemplaza el botón.

microWakeWord corre en el propio ESP32-S3, ~30 kB de RAM, y solo abre el stream cuando detecta la palabra. La CPU de la Pi ni se entera.

Entrenar "Charly":

1. Generá ~1000 muestras sintéticas de la palabra con Piper, variando voz, velocidad y tono.
2. Agregá ruido de fondo de datasets públicos.
3. Entrenás con el pipeline de microWakeWord. Sale un `.tflite` de pocos kB.
4. Lo embebés en el firmware.

Ajustar el umbral: empezá conservador (pocos falsos positivos, algún falso negativo). Un asistente que se despierta solo mientras mirás una película es mucho más molesto que uno al que a veces hay que repetirlo.

Dejá el botón `BOOT` funcionando en paralelo como fallback.

Fase 8 — Gabinete e instalación

Satélite

Caja impresa en 3D o una cajita plástica de proyecto. Requisitos:

- **Agujeros para el micrófono**, alineados con el INMP441. Un solo agujero de 3mm alcanza.
- **Rejilla para el parlante**.

- **Separar el micrófono del parlante** todo lo que la caja permita, o el eco del propio audio te dispara el wake word.
- **Acceso al USB-C.**

Reemplazá la protoboard por una placa perforada soldada. La protoboard funciona para probar, pero los contactos por presión se aflojan con el tiempo y los cambios de temperatura.

Pi

El módulo DAC colgando de seis cables dupont es frágil. Dos opciones:

- **Placa perforada** con header de 40 pines, todo soldado.
- **PCB custom** — solo arriba de ~500 unidades. El NRE no se justifica antes.

Fijá la Pi y el DAC dentro de una misma caja, con el cable RCA saliendo por atrás.

Checklist

#	Fase	Tipo	Tiempo	Estado
0	Baseline y compras	🍓	20 min + envío	☐
1	DAC I2S	🔌 🍓	30 min	☐
2	Matar Bluetooth y medir	🍓	15 min	☐
3	Armar satélite	🔌	1-2 h	☐
4	Firmware push-to-talk	💻 🍓	4-6 h	☐
5	STT en el VPS	⚙️ 🍓	2 h	☐
6	TTS con Piper	🍓 💻	3 h	☐
7	Wake word	💻	4-8 h	☐
8	Gabinete	🔌	variable	☐

Las fases 1 y 2 son una tarde y ya cambian el sistema. La 3 y la 4 son el salto grande. De la 5 en adelante, todo es refinamiento.

Qué mirar en cada fase

Fase 2: el delta de CPU. Es el número que decide si cambiás de SoC o no.

Fase 4: la latencia de punta a punta. Desde que soltás el botón hasta que el texto llega al router de n8n. Objetivo: menos de 2 segundos.

Fase 7: falsos positivos por día. Si es más de uno o dos, subí el umbral aunque pierdas sensibilidad.